

Supplementary Material for A Dirichlet Pólya Inequality for Three-Dimensional Spherical Shells

Certification sources and replay data

15 July 2026

Scope. This supplement accompanies the main proof manuscript. It is not a second analytic proof. It contains the complete exact-rational ledger and the two complete outward-interval certifiers used for the finite residual regions of the spherical-shell argument. All analytic reductions, regional ownership rules, and certificate contracts are stated and proved in the main paper.

Abstract

We print the three executable verification programs used in the proof of the Dirichlet Pólya inequality for three-dimensional spherical shells. The first program checks every finite rational inequality and exact set-ownership identity. The other two programs certify the compact proxy rectangle and the aggregate tail with outward Arb balls. Replay commands, source hashes, certificate hashes, precision gates, and coverage counts are included so that the finite computations can be independently regenerated.

Contents

Relationship to the main paper	2
S1 Standalone exact-rational analytic ledgers	2
S2 Executable interval certificates	18
S2.1 Compact-proxy rectangle certifier	19
S2.2 Aggregate-tail certifier	26
S3 Verification boundary	31

Relationship to the main paper

The main paper contains the theorem statements, the complete shell-specific analytic proof, the definitions of every finite predicate, and the exact regional assembly. Section S1 below supplies the deterministic exact-rational ledger invoked by the seam and staircase arguments. Section S2 supplies the two interval programs invoked by the compact and aggregate certificate lemmas. These listings read no project files. Byte-identical machine-readable copies are distributed as ancillary `.py` files under the displayed filenames; run them with the printed commands.

S1 Standalone exact-rational analytic ledgers

The following program is the complete executable source for the finite arithmetic ledgers used in the seam, two-sided staircase, lower closure, and high staircase arguments. It uses only the Python standard library, reads no file, imports no project module, and has no hidden data input. Every accepted predicate is an integer or rational comparison, an integer-square-root enclosure, or an alternating-series enclosure. Failure of any predicate raises an exception and produces a nonzero exit status.

The ordered registry has two named parts. The core contains 553 checks: 488 substantive inequalities or sign implications and 65 bookkeeping identities. The supplement contains 58 checks that attach the printed seam and two-sided-staircase reserves to their exact wall/cap pairs and record the 227/300 cap-74 incompatibility gap. Thus the canonical registry has 611 rows. Importantly, the number 488 is the count of the combined core ledger; it is not a count of k_{11} -only geometric cells.

The canonicalization is the newline-terminated sequence

```
namespace\tindex\tkind\tlabel.
```

Its digest identifies the ordered predicates and is not an assumption about an external artifact. A successful replay gives

verdict	PASS; first issue: none
core checks	553 = 488 + 65
supplemental checks	58
combined registry	611 rows
canonical registry SHA-256	afd7e054f52aa9a3992fc8ef16d3e755 1586c76bfb589053714f5f55a36792f1
embedded source SHA-256	41691116e0e3e5453b9b209acb93ff78 4724c3a95b83c94bbdf7f6760616e716
seam derivative cap	2260740364246/708624500625 < 16/5
seam derivative margin	6858037754/708624500625
seam initial reserve	2505132463469/2616573970125
lower- d moving-2z lower bound	173863/8427 > 508/27
cap-74 incompatibility gap	227/300

The replay command is simply

Listing 1: Replay command for the exact-rational ledger

```
python verify_analytic_ledgers.py
```

No floating-point display is consulted by a proof predicate. The program checks the finite arithmetic reductions only; the analytic implications that reduce the spectral inequalities to these predicates are proved in the main text.

Listing 2: Complete standalone exact-rational verifier

```

"""Standalone exact-rational verifier for the finite analytic ledgers.

The program reads no files and has no package dependencies. Every accepted
predicate is an integer comparison, a 'fractions.Fraction' comparison, an
integer-square-root enclosure, or an alternating-series enclosure. It
regenerates the seam constants, the two-sided staircase payment tables, the
high-staircase event registry, and the exact ownership truth table. A
canonical SHA-256 of the ordered check registry makes independent replays
directly comparable.

This executable verifies the finite arithmetic reductions. The analytic
steps reducing the spectral problem to these predicates are proved in the
surrounding manuscript and are not silently replaced by computation here.
"""

from __future__ import annotations

from dataclasses import dataclass
from fractions import Fraction as Q
import hashlib
from math import factorial, isqrt
from typing import Callable, Mapping

class VerificationError(AssertionError):
    """First failed exact implication."""

class Ledger:
    def __init__(self) -> None:
        self.labels: list[str] = []
        self.kinds: list[str] = []

    def require(
        self,
        condition: bool,
        label: str,
        detail: str = "",
        *,
        kind: str = "substantive",
    ) -> None:
        if kind not in ("substantive", "bookkeeping", "authentication"):
            raise ValueError(f"unknown check kind: {kind}")
        if not condition:
            suffix = f": {detail}" if detail else ""
            raise VerificationError(f"{label}{suffix}")
        self.labels.append(label)
        self.kinds.append(kind)

    def counts(self) -> dict[str, int]:
        return {
            kind: sum(observed == kind for observed in self.kinds)
            for kind in ("substantive", "bookkeeping", "authentication")
        }

    def canonical_rows(self, namespace: str) -> tuple[str, ...]:
        """Serialize the ordered registry without platform-dependent data."""

        if len(set(self.labels)) != len(self.labels):
            raise VerificationError(f"duplicate labels in {namespace} registry")
        return tuple(
            f"{namespace}\t{index:04d}\t{kind}\t{label}"
            for index, (kind, label) in enumerate(zip(self.kinds, self.labels), 1)
        )

@dataclass(frozen=True)
class Interval:
    """Closed exact rational interval."""

    lo: Q
    hi: Q

    def __post_init__(self) -> None:
        if self.lo > self.hi:
            raise ValueError("reversed interval")

    @staticmethod
    def point(value: int | Q) -> "Interval":
        value = Q(value)
        return Interval(value, value)

    def __add__(self, other: int | Q | "Interval") -> "Interval":
        other = as_interval(other)
        return Interval(self.lo + other.lo, self.hi + other.hi)

    __radd__ = __add__

    def __neg__(self) -> "Interval":
        return Interval(-self.hi, -self.lo)

    def __sub__(self, other: int | Q | "Interval") -> "Interval":
        return self + (-as_interval(other))

    def __rsub__(self, other: int | Q | "Interval") -> "Interval":

```

```

        return as_interval(other) - self

def __mul__(self, other: int | Q | "Interval") -> "Interval":
    other = as_interval(other)
    products = (
        self.lo * other.lo,
        self.lo * other.hi,
        self.hi * other.lo,
        self.hi * other.hi,
    )
    return Interval(min(products), max(products))

__rmul__ = __mul__

def reciprocal(self) -> "Interval":
    if self.lo <= 0 <= self.hi:
        raise ZeroDivisionError("interval contains zero")
    return Interval(Q(1, 1) / self.hi, Q(1, 1) / self.lo)

def __truediv__(self, other: int | Q | "Interval") -> "Interval":
    return self * as_interval(other).reciprocal()

def __rtruediv__(self, other: int | Q | "Interval") -> "Interval":
    return as_interval(other) / self

def __pow__(self, exponent: int) -> "Interval":
    if exponent < 0:
        return (self.reciprocal()) ** (-exponent)
    result = Interval.point(1)
    base = self
    n = exponent
    while n:
        if n & 1:
            result = result * base
        base = base * base
        n //= 2
    return result

def as_interval(value: int | Q | Interval) -> Interval:
    return value if isinstance(value, Interval) else Interval.point(value)

def sqrt_fraction_bounds(value: Q, decimal_digits: int = 50) -> Interval:
    """Certified decimal-grid enclosure of sqrt(value), using integer square roots."""
    if value < 0:
        raise ValueError("negative radicand")
    if value == 0:
        return Interval.point(0)
    scale = 10**decimal_digits
    floor_scaled_square = value.numerator * scale * scale // value.denominator
    root = isqrt(floor_scaled_square)
    lo = Q(root, scale)
    hi = Q(root + 1, scale)
    if not (lo * lo <= value < hi * hi):
        raise VerificationError("integer-square-root enclosure failed")
    return Interval(lo, hi)

def sqrt_interval(value: Interval, decimal_digits: int = 50) -> Interval:
    if value.lo < 0:
        raise ValueError("negative interval radicand")
    return Interval(
        sqrt_fraction_bounds(value.lo, decimal_digits).lo,
        sqrt_fraction_bounds(value.hi, decimal_digits).hi,
    )

def atan_reciprocal_bounds(q: int, terms: int) -> Interval:
    """Alternating-series enclosure for atan(1/q)."""
    x = Q(1, q)
    total = Q(0)
    for k in range(terms):
        term = x ** (2 * k + 1) / (2 * k + 1)
        total += term if k % 2 == 0 else -term
    next_term = x ** (2 * terms + 1) / (2 * terms + 1)
    if terms % 2 == 0: # next sign is positive
        return Interval(total, total + next_term)
    return Interval(total - next_term, total)

def pi_bounds() -> Interval:
    """Machin: pi = 16 atan(1/5) - 4 atan(1/239)."""
    a = atan_reciprocal_bounds(5, 36)
    b = atan_reciprocal_bounds(239, 12)
    return 16 * a - 4 * b

PI = pi_bounds()

def verify_pi(ledger: Ledger) -> None:
    ledger.require(PI.lo < PI.hi, "Machin interval is nonempty")
    ledger.require(PI.lo > Q(333, 106), "pi > 333/106")

```

```

ledger.require(PI.hi < Q(22, 7), "pi < 22/7")
ledger.require(Q(106, 333) * PI.lo > 1, "1/pi < 106/333")

def exact_constants() -> dict[str, Interval]:
    sqrt2 = sqrt_fraction_bounds(Q(2))
    sqrt3 = sqrt_fraction_bounds(Q(3))
    sqrt337 = sqrt_fraction_bounds(Q(337))
    omega = sqrt3 / (2 * PI) - Q(1, 6)
    c_star = Q(1, 2) + Q(8, 15) * sqrt2
    c0 = 1 + Q(8, 15) * sqrt2
    rho_star = omega / (1 + 2 * c_star)
    rho0 = 1 / sqrt337
    sigma = Q(3, 4) * omega
    rho_c = 1 / (1 + 2 * PI)
    d = sqrt337 / 2
    return {
        "sqrt2": sqrt2,
        "sqrt3": sqrt3,
        "sqrt337": sqrt337,
        "omega": omega,
        "Cstar": c_star,
        "C0": c0,
        "rho_star": rho_star,
        "rho0": rho0,
        "sigma": sigma,
        "rho_c": rho_c,
        "d": d,
    }

def separated(left: Interval, right: Interval) -> bool:
    return left.hi < right.lo

def verify_thresholds(ledger: Ledger) -> dict[str, Interval]:
    c = exact_constants()
    chain: list[tuple[str, Interval]] = [
        ("rho_0", c["rho_star"]),
        ("rho_0", c["rho0"]),
        ("sigma", c["sigma"]),
        ("rho_c", c["rho_c"]),
        ("39/50", Interval.point(Q(39, 50))),
        ("7/8", Interval.point(Q(7, 8))),
    ]
    for (lname, left), (rname, right) in zip(chain, chain[1:]):
        ledger.require(separated(left, right), f"threshold order {lname} < {rname}")

    ledger.require(separated(c["omega"], c["rho_c"]), "omega_0 < rho_c (H0 ineligible)")
    ledger.require(
        (c["omega"] * c["d"]).lo > 1,
        "omega_0 d > 1",
    )

    sqrt114 = sqrt_fraction_bounds(Q(114))
    c42 = sqrt_fraction_bounds(Q(7073, 64))
    fixed = [
        ("d", c["d"]),
        ("10", Interval.point(10)),
        ("81/8", Interval.point(Q(81, 8))),
        ("21/2", Interval.point(Q(21, 2))),
        ("sqrt(7073)/8", c42),
        ("sqrt(114)", sqrt114),
    ]
    for (lname, left), (rname, right) in zip(fixed, fixed[1:]):
        ledger.require(separated(left, right), f"lower-frequency order {lname} < {rname}")
    ledger.require(Q(7073, 64) == Q(81, 8) ** 2 + 8, "c42 propagation identity")

    z_at_rho0 = PI / (1 - c["rho0"])
    z_at_rho_c = PI + Q(1, 2) # exact simplification at rho_c
    ledger.require(separated(c["d"], 3 * z_at_rho0), "d < 3 z(rho0)")
    ledger.require((3 * z_at_rho0).hi < 10, "3 z(rho0) < 10")
    ledger.require(separated(sqrt114, 3 * z_at_rho_c), "sqrt(114) < 3 z(rho_c)")

    moving_walls = [
        ("10", Interval.point(10)),
        ("81/8", Interval.point(Q(81, 8))),
        ("21/2", Interval.point(Q(21, 2))),
        ("c42", c42),
        ("sqrt114", sqrt114),
    ]
    crossings: list[tuple[str, Interval]] = []
    for name, wall in moving_walls:
        crossing = 1 - 3 * PI / wall
        ledger.require(separated(c["rho0"], crossing), f"3z={name} crossing above rho0")
        ledger.require(separated(crossing, c["rho_c"]), f"3z={name} crossing below rho_c")
        crossings.append((name, crossing))
    for (lname, left), (rname, right) in zip(crossings, crossings[1:]):
        ledger.require(separated(left, right), f"moving-wall order {lname} before {rname}")

    r_proxy = Interval.point(Q(367, 20))
    ledger.require(separated(sqrt114, 2 / c["omega"]), "sqrt(114) < 2/omega_0")
    ledger.require(separated(2 / c["omega"], r_proxy), "2/omega_0 < 367/20")
    ledger.require(
        separated(c["rho_c"], Q(3, 2) * c["omega"]),
        "rho_c < 3 omega_0/2",
    )

```

```

    ledger.require((c["omega"] * sqrt(114)).lo > 1, "omega_0 sqrt(114) > 1")
    return {**c, "sqrt114": sqrt(114), "c42": c42, "R": r_proxy}

def ceil_q(value: Q) -> int:
    return -((-value.numerator) // value.denominator)

def floor_q(value: Q) -> int:
    return value.numerator // value.denominator

def strict_positive_integer_count(value: Q) -> int:
    return max(0, ceil_q(value) - 1)

def verify_strict_face_conventions(ledger: Ledger) -> None:
    for m in range(1, 9):
        ledger.require(
            strict_positive_integer_count(Q(m)) == m - 1,
            f"strict integer bracket at {m}",
        )
        ledger.require(
            strict_positive_integer_count(Q(2 * m + 1, 2)) == m,
            f"strict integer bracket between walls {m}",
        )
    # If  $x^2 = ell(ell+1)$ ,  $y = \sqrt{x^2 + 1/4} - 1/2 = ell$  exactly;
    #  $M = ceil(y) = ell$  counts  $ell=0, \dots, ell-1$  and excludes the equality channel.
    for ell in range(0, 12):
        ledger.require(
            sum(2 * j + 1 for j in range(ell)) == ell * ell,
            f"product angular equality excludes ell={ell}",
        )
    # At  $A+1/4=n$ ,  $[A+1/4]_- = n-1$  and the layer  $x_n = n-1/4$  is excluded.
    for n in range(1, 8):
        ledger.require(
            strict_positive_integer_count(Q(n)) == n - 1,
            f"phase-proxy integer wall n={n}",
        )

def taylor_sin_cos(value: Q, terms: int = 70) -> tuple[Interval, Interval]:
    """Global Taylor enclosures; derivative remainders are bounded by one."""
    sin_sum = Q(0)
    cos_sum = Q(0)
    for k in range(terms):
        sin_term = value ** (2 * k + 1) / factorial(2 * k + 1)
        cos_term = value ** (2 * k) / factorial(2 * k)
        sin_sum += sin_term if k % 2 == 0 else -sin_term
        cos_sum += cos_term if k % 2 == 0 else -cos_term
    sin_err = abs(value) ** (2 * terms) / factorial(2 * terms)
    cos_err = abs(value) ** (2 * terms - 1) / factorial(2 * terms - 1)
    return (
        Interval(sin_sum - sin_err, sin_sum + sin_err),
        Interval(cos_sum - cos_err, cos_sum + cos_err),
    )

def spherical_coefficients(ell: int, x: Q) -> tuple[Q, Q]:
    """Return A,B with spherical_j_ell(x)=A sin(x)+B cos(x)."""
    if x <= 0:
        raise ValueError("x must be positive")
    a0, b0 = Q(1, 1) / x, Q(0)
    if ell == 0:
        return a0, b0
    a1, b1 = Q(1, 1) / (x * x), -Q(1, 1) / x
    if ell == 1:
        return a1, b1
    prev = (a0, b0)
    cur = (a1, b1)
    for l in range(1, ell):
        nxt = (
            Q(2 * l + 1, 1) * cur[0] / x - prev[0],
            Q(2 * l + 1, 1) * cur[1] / x - prev[1],
        )
        prev, cur = cur, nxt
    return cur

def linear_interval(a: Q, x: Interval, b: Q, y: Interval) -> Interval:
    return a * x + b * y

def spherical_sign_at_rational(ell: int, x: Q) -> int:
    a, b = spherical_coefficients(ell, x)
    sin_x, cos_x = taylor_sin_cos(x)
    value = linear_interval(a, sin_x, b, cos_x)
    if value.lo > 0:
        return 1
    if value.hi < 0:
        return -1
    raise VerificationError(f"unresolved exact spherical-Bessel sign ell={ell}, x={x}")

def interval_spherical_coefficients(ell: int, x: Interval) -> tuple[Interval, Interval]:

```

```

a0, b0 = 1 / x, Interval.point(0)
if ell == 0:
    return a0, b0
a1, b1 = 1 / (x * x), -(1 / x)
if ell == 1:
    return a1, b1
prev = (a0, b0)
cur = (a1, b1)
for l in range(1, ell):
    nxt = (
        (2 * l + 1) * cur[0] / x - prev[0],
        (2 * l + 1) * cur[1] / x - prev[1],
    )
    prev, cur = cur, nxt
return cur

def spherical_sign_at_half_period(ell: int, half_index: int) -> int:
    x = Q(half_index, 2) * PI
    a, b = interval_spherical_coefficients(ell, x)
    phase = half_index % 4
    sin_value = (0, 1, 0, -1)[phase]
    cos_value = (1, 0, -1, 0)[phase]
    value = sin_value * a + cos_value * b
    if value.lo > 0:
        return 1
    if value.hi < 0:
        return -1
    raise VerificationError(
        f"unresolved half-period sign ell={ell}, half_index={half_index}"
    )

@dataclass(frozen=True)
class InternalZeroFace:
    ell: int
    radial: int
    wall: Q
    expected_sign: int
    left_half: int
    right_half: int
    left_sign: int
    right_sign: int

INTERNAL_ZERO_FACES = (
    InternalZeroFace(1, 2, Q(77, 10), -1, 4, 5, -1, 1),
    InternalZeroFace(2, 2, Q(9), -1, 5, 6, -1, 1),
    InternalZeroFace(3, 2, Q(103, 10), -1, 6, 7, -1, 1),
    InternalZeroFace(5, 2, Q(129, 10), -1, 8, 9, -1, 1),
    InternalZeroFace(1, 3, Q(21, 2), 1, 6, 7, 1, -1),
    InternalZeroFace(2, 3, Q(61, 5), 1, 7, 8, 1, -1),
    InternalZeroFace(6, 1, Q(10), 1, 6, 7, 1, -1),
    InternalZeroFace(7, 1, Q(23, 2), 1, 7, 8, 1, -1),
)

LORCH_ZERO_FACES = (
    (8, Q(71, 6)),
    (9, Q(64, 5)),
    (10, Q(69, 5)),
)

def lorch_l2_rhs(ell: int) -> Interval:
    nu = Q(2 * ell + 1, 2)
    radical = sqrt_fraction_bounds((2 * nu + 3) * (2 * nu + 11))
    denominator = 1 - 2 * nu + radical
    if denominator.lo <= 0:
        raise VerificationError(f"Lorch L2 denominator not positive for ell={ell}")
    return 24 * (nu + 1) ** 2 / denominator - 2 * (nu * nu - 1)

def verify_zero_specializations(ledger: Ledger) -> dict[tuple[int, int], Q]:
    registry: dict[tuple[int, int], Q] = {}
    for face in INTERNAL_ZERO_FACES:
        ledger.require(
            Q(face.left_half, 2) * PI).hi < face.wall,
            f"zero wall ell={face.ell},n={face.radial} right of left half-period",
        )
        ledger.require(
            face.wall < (Q(face.right_half, 2) * PI).lo,
            f"zero wall ell={face.ell},n={face.radial} left of right half-period",
        )
        ledger.require(
            spherical_sign_at_half_period(face.ell, face.left_half) == face.left_sign,
            f"left tangent-cell sign ell={face.ell},n={face.radial}",
        )
        ledger.require(
            spherical_sign_at_half_period(face.ell, face.right_half) == face.right_sign,
            f"right tangent-cell sign ell={face.ell},n={face.radial}",
        )
        ledger.require(
            spherical_sign_at_rational(face.ell, face.wall) == face.expected_sign,
            f"rational zero-wall sign ell={face.ell},n={face.radial}",
        )
    registry[(face.ell, face.radial)] = face.wall * face.wall

```

```

for ell, wall in LORCH_ZERO_FACES:
    rhs = lorch_l2_rhs(ell)
    ledger.require(
        rhs.lo > wall * wall,
        f"Lorch L2 exact specialization ell={ell}",
    )
    ledger.require(
        spherical_sign_at_rational(ell, wall) == 1,
        f"first-zero rational-wall sign ell={ell}",
    )
    registry[(ell, 1)] = wall * wall

ledger.require(Q(103, 10) > Q(81, 8), "strong j_7/2,2 wall implies 81/8 wall")
ledger.require(Q(61, 5) > Q(21, 2), "third-mode wall order 21/2 < 61/5")
return registry

def angular_sum(max_ell: int) -> int:
    return sum(2 * ell + 1 for ell in range(max_ell + 1))

def weyl_interval(rho: Interval, k: Interval) -> Interval:
    return Q(2, 9) * (1 / PI) * (1 - rho**3) * k**3

def verify_lower_inventories_and_payments(
    ledger: Ledger, constants: Mapping[str, Interval]
) -> None:
    # Exclusion/activation arithmetic. Strict zero walls are analytic inputs;
    # all consequences at included K faces are checked here.
    ledger.require(Q(23, 2) ** 2 > 114, "first ell=7 absent through sqrt114")
    ledger.require(Q(10) ** 2 == 100, "first ell=6 activation wall 10")
    ledger.require(Q(129, 10) ** 2 > 114, "second ell=5 absent through sqrt114")
    ledger.require(Q(103, 10) > Q(81, 8), "second ell=3 absent through 81/8")
    ledger.require(
        Q(81, 8) ** 2 + 8 == Q(7073, 64),
        "second ell=4 propagated c42 wall",
    )
    ledger.require(
        Q(81, 8) ** 2 + 18 > 114,
        "second ell=5 propagated absence",
    )
    ledger.require(Q(21, 2) ** 2 + 4 > 114, "third ell=2 propagated absence")
    ledger.require((16 * PI**2).lo > 114, "all n>=4 absent through sqrt114")

    expected_caps = (45, 46, 59, 66, 69, 78)
    derived_caps = (
        angular_sum(5) + angular_sum(2),
        angular_sum(5) + angular_sum(2) + 1,
        angular_sum(6) + angular_sum(2) + 1,
        angular_sum(6) + angular_sum(3) + 1,
        angular_sum(6) + angular_sum(3) + 1 + 3,
        angular_sum(6) + angular_sum(4) + 1 + 3,
    )
    ledger.require(
        derived_caps == expected_caps,
        "all six lower inventory caps",
        kind="bookkeeping",
    )

    rho_c = constants["rho_c"]
    payments = (
        (constants["d"], 46, "d"),
        (Interval.point(10), 59, "10"),
        (Interval.point(Q(81, 8)), 66, "81/8"),
        (Interval.point(Q(21, 2)), 69, "21/2"),
        (constants["c42"], 78, "c42"),
    )
    for k, cap, name in payments:
        reserve = weyl_interval(rho_c, k)
        ledger.require(reserve.lo > cap, f"lower Weyl payment above {name} pays cap {cap}")

    proxy_caps = (6, 5, 5, 4, 4, 3, 3, 3, 2, 2, 1, 1, 1, 1)
    ledger.require(
        sum((2 * ell + 1) * cap for ell, cap in enumerate(proxy_caps)) == 395,
        "exceptional-cell proxy cap is exactly 395",
    )
    r = Q(367, 20)
    for ell, cap in enumerate(proxy_caps + (0,)):
        nu = Q(2 * ell + 1, 2)
        radical_1 = sqrt_fraction_bounds(r * r - nu * nu)
        radical_2 = sqrt_fraction_bounds(2 * (1 - nu / r))
        # arccos(x) >= sqrt(2(1-x)); hence this is an exact upper bound for G_R.
        numerator_upper = radical_1.hi - nu * radical_2.lo
        proxy_upper = Interval.point(numerator_upper) / PI + Q(1, 4)
        ledger.require(
            proxy_upper.hi < cap + 1,
            f"strict phase proxy ell={ell} has cap {cap}",
        )
    # G_R is decreasing in nu; ell=14 therefore kills every omitted angular tail.
    tail_caps = proxy_caps + (0,)
    ledger.require(
        tail_caps[-1] == 0 and all(a >= b for a, b in zip(tail_caps, tail_caps[1:])),
        "ell>=14 omitted by monotonicity after exact ell=14 check",
    )

```



```

rho_c_k = Q(5, 2) / rho_c
exceptional_payment = weyl_interval(rho_c, rho_c_k)
ledger.require(exceptional_payment.lo > 395, "exceptional cell Weyl payment exceeds 395")

# Exact floor-cell logic. p>=1 because omega*K>1. If p>=2, then
# rho*K < (3/2)(p+1) <= 2p+1/2, with strictness at p=2.
ledger.require(Q(3, 2) * 3 == Q(9, 2), "p=2 floor-cell endpoint identity")
for p in range(2, 10):
    ledger.require(
        Q(3, 2) * (p + 1) <= 2 * p + Q(1, 2),
        f"floor-cell inequality p={p}",
    )
ledger.require(
    (Q(5, 2) - Q(1, 2)).denominator == 1,
    "rho K=5/2 is assigned to failing high half-integer side",
)
normalized_face = Q(2) # omega_0 K at K=2/omega_0
endpoint_rho_k = 2 * constants["rho_c"] / constants["omega"]
ledger.require(
    floor_q(normalized_face) == 2 and endpoint_rho_k.hi < 3,
    "K=2/omega has p=2 and rho*K<3",
)
ledger.require(
    endpoint_rho_k.hi - Q(1, 2) < Q(5, 2) and 2 <= 2 * floor_q(normalized_face) - 1,
    "K=2/omega satisfies m<=2p-1 and belongs to the complement",
)

def propagated_lower_square(
    bases: Mapping[int, Q], target_ell: int
) -> Q | None:
    candidates = [
        square + target_ell * (target_ell + 1) - ell * (ell + 1)
        for ell, square in bases.items()
        if ell <= target_ell
    ]
    return max(candidates) if candidates else None

FIRST_BASES = {
    6: Q(10) ** 2,
    7: Q(23, 2) ** 2,
    8: Q(71, 6) ** 2,
    9: Q(64, 5) ** 2,
    10: Q(69, 5) ** 2,
}
SECOND_BASES = {
    1: Q(77, 10) ** 2,
    2: Q(9) ** 2,
    3: Q(103, 10) ** 2,
    5: Q(129, 10) ** 2,
}
THIRD_BASES = {1: Q(21, 2) ** 2, 2: Q(61, 5) ** 2}

def verify_zero_channel_cover(
    ledger: Ledger,
    checkpoint: int,
    radial: int,
    first_excluded_ell: int,
    bases: Mapping[int, Q],
) -> None:
    mterm = checkpoint * (checkpoint + 1)
    ell = first_excluded_ell
    channel_cut = Q(mterm - ell * (ell + 1), radial * radial - 1)
    zero_sq = propagated_lower_square(bases, ell)
    if zero_sq is None:
        raise VerificationError("missing zero base")
    zero_cut = zero_sq - mterm
    ledger.require(
        zero_cut >= channel_cut,
        f"zero/channel cover k{checkpoint}, n={radial}, ell>={ell}",
        f"zero cut {zero_cut}, channel cut {channel_cut}",
    )

def x_at_ratio(rho: Q) -> Interval:
    return PI**2 / (1 - rho) ** 2

def checkpoint_weyl(rho: Interval, checkpoint: int) -> Interval:
    x = PI**2 / (1 - rho) ** 2
    k = sqrt_interval(x + checkpoint * (checkpoint + 1))
    return weyl_interval(rho, k)

def conditional_first_mode_payment(
    ledger: Ledger, checkpoint: int, wall: Q, cap: int, label: str
) -> None:
    mterm = checkpoint * (checkpoint + 1)
    x0 = wall * wall - mterm
    ledger.require(x0 > 0, f"{label} positive localization x")
    root_x = sqrt_fraction_bounds(x0)
    rho0 = 1 - PI / root_x
    payment = weyl_interval(rho0, Interval.point(wall))
    ledger.require(payment.lo > cap, f"{label} conditional payment > {cap}")

```

```

def verify_k9_h6_h4_channel_equality(
    ledger: Ledger,
    channel_wall: Q,
) -> Interval:
    """Require that the supplied wall is exactly the k9 (ell,n)=(4,2) wall."""

    prefix = "k9 H=6,h=4 cap74"
    delta = channel_delta(9, 2, 4, Interval.point(channel_wall))
    ledger.require(
        delta.lo == 0 and delta.hi == 0,
        f"{prefix}: z^2=70/3 is the defining k9 ell=4,n=2 equality",
    )
    return delta

def verify_k9_h6_h4_path(
    ledger: Ledger,
    *,
    channel_wall: Q = Q(70, 3),
) -> dict[str, Q | Interval]:
    """Close the previously missing k9 H=6,h=4 cap-74 payment cell."""

    prefix = "k9 H=6,h=4 cap74"
    base_square = Q(103, 10) ** 2
    propagated_square = base_square + (4 * 5 - 3 * 4)
    ledger.require(
        propagated_square == Q(11409, 100),
        f"{prefix}: ell=3 to ell=4 n=2 propagation square",
    )
    ledger.require(
        propagated_square > 108,
        f"{prefix}: propagated second-mode square exceeds 108",
    )

    equality_delta = verify_k9_h6_h4_channel_equality(ledger, channel_wall)
    ledger.require(
        strict_positive_integer_count(Q(1)) == 0,
        f"{prefix}: strict count excludes the defining equality mode",
    )

    rho_face = Q(7, 20)
    x_face = x_at_ratio(rho_face)
    ledger.require(
        x_face.lo > channel_wall,
        f"{prefix}: rho=7/20 lies on excluded side z^2>70/3",
    )
    ledger.require(
        Q(400, 169) * Q(333, 106) ** 2 - channel_wall
        == Q(36230, 1424163),
        f"{prefix}: exact rational reserve at rho=7/20",
    )
    ledger.require(
        x_face.lo > channel_wall and strict_positive_integer_count(Q(1)) == 0,
        f"{prefix}: a counted ell=4 second mode forces rho<7/20",
    )

    frequency_wall = Q(207, 20)
    ledger.require(
        frequency_wall**2 == Q(42849, 400),
        f"{prefix}: exact (207/20)^2",
    )
    ledger.require(
        frequency_wall**2 < 108 < propagated_square,
        f"{prefix}: counted ell=4 second mode forces K>207/20",
    )

    payment = weyl_interval(Interval.point(rho_face), Interval.point(frequency_wall))
    coarse_payment = (
        Q(2, 9)
        * Q(7, 22)
        * (1 - rho_face**3)
        * frequency_wall**3
    )
    ledger.require(
        coarse_payment == Q(52823261673, 704000000),
        f"{prefix}: exact pi<22/7 Weyl lower payment",
    )
    ledger.require(
        coarse_payment - 74 == Q(727261673, 704000000) > 0,
        f"{prefix}: exact Weyl reserve above cap 74",
    )
    ledger.require(
        payment.lo > coarse_payment > 74,
        f"{prefix}: certified W(7/20,207/20)>74",
    )
    ledger.require(
        49 + 25 == 74,
        f"{prefix}: full first and second multiplicities equal cap 74",
        kind="bookkeeping",
    )
    return {
        "base_square": base_square,
        "propagated_square": propagated_square,
        "channel_wall": channel_wall,
        "equality_delta": equality_delta,
        "rho_face": rho_face,
    }

```

```

    "frequency_wall": frequency_wall,
    "coarse_payment": coarse_payment,
    "payment": payment,
}

def verify_checkpoint_inventories(ledger: Ledger, constants: Mapping[str, Interval]) -> None:
    expected = {
        7: (6, 1, -1, 2),
        8: (7, 3, -1, 2),
        9: (8, 4, -1, 2),
        10: (9, 5, 1, 3),
        11: (10, 4, 1, 3),
    }
    for m, (first_max, second_max, third_max, max_radial) in expected.items():
        ledger.require(first_max == m - 1, f"k{m} strict first-mode upper face")
        for ell in range(m, 12):
            ledger.require(
                ell * (ell + 1) >= m * (m + 1),
                f"k{m} first angular tail excluded at ell={ell}",
            )
        ledger.require(second_max < m, f"k{m} finite second inventory")
        if third_max >= 0:
            ledger.require(third_max == 1, f"k{m} third inventory ell=0,1")
        ledger.require(max_radial in (2, 3), f"k{m} stated maximal radial index")

    verify_zero_channel_cover(ledger, 7, 2, 2, SECOND_BASES)
    verify_zero_channel_cover(ledger, 8, 2, 4, SECOND_BASES)
    verify_zero_channel_cover(ledger, 9, 2, 5, SECOND_BASES)
    verify_zero_channel_cover(ledger, 10, 2, 6, SECOND_BASES)
    verify_zero_channel_cover(ledger, 11, 2, 5, SECOND_BASES)

    x_min = (PI + Q(1, 2)) ** 2
    ledger.require(x_min.lo > Q(90, 8), "no third modes at k7,k8,k9")
    verify_zero_channel_cover(ledger, 10, 3, 2, THIRD_BASES)
    verify_zero_channel_cover(ledger, 11, 3, 2, THIRD_BASES)
    ledger.require(x_min.lo > Q(132, 15), "no radial n>=4 through k11")

    # Every omitted angular tail is monotone: the propagated zero cut increases
    # and the channel cut decreases with ell. The base-ell cover therefore
    # proves all remaining indices, not merely ell<=11.
    for m, first_ell in ((7, 2), (8, 4), (9, 5), (10, 6), (11, 5)):
        M = m * (m + 1)
        previous_zero = None
        previous_channel = None
        for ell in range(first_ell, 13):
            zcut = propagated_lower_square(SECOND_BASES, ell) - M # type: ignore[operator]
            ccut = Q(M - ell * (ell + 1), 3)
            if previous_zero is not None:
                ledger.require(zcut > previous_zero, f"k{m} zero cut increases at ell={ell}")
                ledger.require(ccut < previous_channel, f"k{m} channel cut decreases at ell={ell}")
            previous_zero, previous_channel = zcut, ccut

def verify_cap_tables_and_payments(ledger: Ledger, constants: Mapping[str, Interval]) -> None:
    # k7
    ledger.require(
        (25, 36, 49) == tuple((h + 1) ** 2 for h in (4, 5, 6)),
        "k7 first caps",
        kind="bookkeeping",
    )
    ledger.require(1 + 3 == 4, "k7 two second modes add 1+3", kind="bookkeeping")

    # k8 and k9 rectangular tables: every numerical entry is exactly the sum
    # of a full first block and a full second block.
    k8 = {
        7: (64, 65, 68, 73, 80),
        6: (49, 50, 53, None, None),
        5: (36, 37, 40, 45, 52),
        4: (25, 26, 29, 34, 41),
    }
    for h_first, row in k8.items():
        for col, value in enumerate(row):
            if value is not None:
                expected = (h_first + 1) ** 2 + (0 if col == 0 else col * col)
                ledger.require(
                    value == expected,
                    f"k8 cap H={h_first}, column={col}",
                    kind="bookkeeping",
                )

    k9 = {
        8: (81, None, None, None, None, None),
        7: (64, 65, 68, 73, None, None),
        6: (49, 50, 53, 58, 65, 74),
        5: (36, 37, 40, 45, 52, 61),
        4: (25, 26, 29, 34, 41, 50),
    }
    for h_first, row in k9.items():
        for col, value in enumerate(row):
            if value is not None:
                expected = (h_first + 1) ** 2 + (0 if col == 0 else col * col)
                ledger.require(
                    value == expected,
                    f"k9 cap H={h_first}, column={col}",
                    kind="bookkeeping",
                )

```

```

verify_k9_h6_h4_path(ledger)

# Marked impossible cells, proved from simultaneous strict necessary walls.
ledger.require(Q(100 - 72) > Q(72 - 6, 3), "k8 H=6 with h>=2 impossible")
ledger.require(Q(71, 6) ** 2 - 90 > Q(90, 3), "k9 H=8 with any second impossible")
ledger.require(Q(23, 2) ** 2 - 90 > Q(90 - 12, 3), "k9 H=7 with h>=3 impossible")

k10_cases = (
    (100, 100, "H9 no radial"),
    (81, 81, "H8 no second"),
    (81 + 16, 97, "H8 h<=3"),
    (64, 64, "H7 no second"),
    (64 + 16, 80, "H7 h<=3"),
    (64 + 25, 89, "H7 h=4"),
    (64 + 36, 100, "H7 h=5"),
    (49 + 25, 74, "H<=6 h<=4 no third"),
    (49 + 16 + 4, 69, "H<=6 h<=3 third"),
    (49 + 25 + 4, 78, "H<=6 h=4 third"),
    (49 + 36, 85, "H<=6 h=5 no third"),
    (49 + 36 + 4, 89, "H<=6 h=5 third"),
)
for derived, frozen, name in k10_cases:
    ledger.require(derived == frozen, f"k10 cap {name}", kind="bookkeeping")

k11_cases = (
    (121, 121, "H10 no radial"),
    (100, 100, "H9 no radial"),
    (100 + 25, 125, "H9 second no third"),
    (81 + 25, 106, "H8 no third"),
    (81 + 25 + 4, 110, "H8 third"),
    (64 + 25 + 4, 93, "H<=7 all radial"),
)
for derived, frozen, name in k11_cases:
    ledger.require(derived == frozen, f"k11 cap {name}", kind="bookkeeping")

# W_m is increasing because pi^2(1+rho) > M rho^2(1-rho)^2;
# the exact uniform reduction is pi^2>9>132/16.
ledger.require(Pi.lo**2 > 9, "checkpoint-Weyl derivative: pi^2>9")
ledger.require(Q(132, 16) < 9, "checkpoint-Weyl derivative: M/16<9")

rho_c = constants["rho_c"]
global_payments = ((7, 40), (8, 53), (9, 73), (10, 89), (11, 121))
for m, cap in global_payments:
    ledger.require(
        checkpoint_weyl(rho_c, m).lo > cap,
        f"global k[{m}] checkpoint payment > {cap}",
    )

conditional_first_mode_payment(ledger, 7, Q(10), 53, "k7 H6")
conditional_first_mode_payment(ledger, 8, Q(23, 2), 80, "k8 H7")
conditional_first_mode_payment(ledger, 9, Q(71, 6), 81, "k9 H8")
conditional_first_mode_payment(ledger, 10, Q(23, 2), 100, "k10 H7")
conditional_first_mode_payment(ledger, 10, Q(71, 6), 97, "k10 H8")
conditional_first_mode_payment(ledger, 10, Q(64, 5), 100, "k10 H9")
conditional_first_mode_payment(ledger, 11, Q(64, 5), 125, "k11 H9")

LOCALIZATION_RATIOS = (
    Q(1, 5), Q(3, 10), Q(1, 3), Q(3, 8), Q(2, 5), Q(5, 12), Q(3, 7),
    Q(1, 2), Q(107, 200), Q(8, 15), Q(3, 5), Q(16, 25), Q(13, 20), Q(2, 3),
    Q(4, 25), Q(1, 4), Q(7, 20),
)

@dataclass(frozen=True)
class LocalizationExpectation:
    rho: Q
    checkpoint: int
    radial: int
    ell: int
    expected_sign: int
    purpose: str

LOCALIZATION_EXPECTATIONS = (
    LocalizationExpectation(Q(1, 5), 11, 3, 1, 1, "k11 third-mode low-side cell"),
    LocalizationExpectation(Q(3, 10), 8, 2, 3, -1, "k8 h=3 exclusion side"),
    LocalizationExpectation(Q(1, 3), 8, 2, 2, -1, "k8 h=2 exclusion side"),
    LocalizationExpectation(Q(3, 8), 9, 2, 3, 1, "k9 h=3 possible side"),
    LocalizationExpectation(Q(2, 5), 9, 2, 2, 1, "k9 h=2 possible side"),
    LocalizationExpectation(Q(5, 12), 9, 2, 1, 1, "k9 h=1 possible side"),
    LocalizationExpectation(Q(3, 7), 9, 2, 0, -1, "k9 second-mode exclusion side"),
    LocalizationExpectation(Q(1, 2), 10, 2, 0, -1, "k10 second-mode exclusion side"),
    LocalizationExpectation(Q(107, 200), 11, 2, 0, -1, "k11 second-mode exclusion side"),
    LocalizationExpectation(Q(8, 15), 11, 2, 0, -1, "k11 stated second-mode localization"),
    LocalizationExpectation(Q(3, 5), 8, 2, 0, -1, "k8 all-second exclusion/payment side"),
    LocalizationExpectation(Q(16, 25), 9, 2, 0, -1, "k9 all-second exclusion/payment side"),
    LocalizationExpectation(Q(13, 20), 10, 2, 0, -1, "k10 all-second exclusion/payment side"),
    LocalizationExpectation(Q(2, 3), 11, 2, 0, -1, "k11 all-second exclusion/payment side"),
    LocalizationExpectation(Q(4, 25), 10, 3, 0, -1, "k10 third-mode exclusion side"),
    LocalizationExpectation(Q(1, 4), 11, 3, 0, -1, "k11 stated third-mode localization"),
    LocalizationExpectation(Q(7, 20), 9, 2, 4, -1, "k9 H=6,h=4 cap74 payment side"),
)

def channel_delta(

```

```

    checkpoint: int,
    radial: int,
    ell: int,
    x: Interval,
) -> Interval:
    """Exact channel difference  $M - \text{ell}(\text{ell}+1) - (n^2-1) z^2$ ."""

    return (
        Interval.point(checkpoint * (checkpoint + 1) - ell * (ell + 1))
        - (radial * radial - 1) * x
    )

def verify_localization_expectation(
    ledger: Ledger,
    item: LocalizationExpectation,
    *,
    x_override: Interval | None = None,
) -> Interval:
    """Verify one named side, with injectable  $x$  for semantic mutation tests."""

    x = x_at_ratio(item.rho) if x_override is None else x_override
    delta = channel_delta(item.checkpoint, item.radial, item.ell, x)
    condition = delta.lo > 0 if item.expected_sign > 0 else delta.hi < 0
    side = "possible" if item.expected_sign > 0 else "excluded"
    ledger.require(
        condition,
        f"rho={item.rho} expected {side} side for {item.purpose}",
    )
    return delta

def verify_localizations(ledger: Ledger, constants: Mapping[str, Interval]) -> None:
    ratios = sorted(set(LOCALIZATION_RATIOS))
    ledger.require(
        len(ratios) == len(LOCALIZATION_RATIOS),
        "localization ratios are distinct",
        kind="bookkeeping",
    )
    ledger.require(
        {item.rho for item in LOCALIZATION_EXPECTATIONS} == set(LOCALIZATION_RATIOS),
        "every named ratio has a connected expected-side assertion",
        kind="bookkeeping",
    )
    for rho in ratios:
        ledger.require(constants["rho_c"].hi < rho < Q(7, 8), f"ratio localization {rho} in high strip")

    for item in LOCALIZATION_EXPECTATIONS:
        verify_localization_expectation(ledger, item)

    x_min = (PI + Q(1, 2)) ** 2
    x_max = 64 * PI**2
    for x in (Q(16), Q(68, 3), Q(34)):
        ledger.require(x_min.hi < x < x_max.lo, f"algebraic split  $z^2=\{x\}$  in high strip")
    ledger.require(
        132 - 1 * 2 - 8 * Q(16) == 2 > 0,
        " $z^2=16$  is on the possible side of  $k_{11} n_3 \text{ ell}_1$ ",
    )
    ledger.require(
        110 - 6 * 7 - 3 * Q(68, 3) == 0
        and strict_positive_integer_count(Q(1)) == 0,
        " $z^2=68/3$  defines and strictly excludes  $k_{10} n_2 \text{ ell}_6$  equality",
    )
    ledger.require(
        132 - 5 * 6 - 3 * Q(34) == 0
        and strict_positive_integer_count(Q(1)) == 0,
        " $z^2=34$  defines and strictly excludes  $k_{11} n_2 \text{ ell}_5$  equality",
    )

    # Explicit  $k_{11}$  localizations.
    ledger.require(x_at_ratio(Q(8, 15)).lo > 44, "k11 second mode forces rho<8/15")
    ledger.require(x_at_ratio(Q(1, 4)).lo > Q(33, 2), "k11 third mode forces rho<1/4")
    h10_lower = Q(69, 5) ** 2 - 132
    h9_lower = Q(64, 5) ** 2 - 132
    ledger.require(h10_lower > 44, "k11 H10 excludes every second mode")
    ledger.require(h10_lower > Q(33, 2), "k11 H10 excludes every third mode")
    ledger.require(h9_lower > Q(33, 2), "k11 H9 excludes every third mode")

def verify_u_and_k11(ledger: Ledger, constants: Mapping[str, Interval]) -> None:
    #  $K_0$  is the positive root of  $\eta a^K - \sqrt{a^K} - C_0 = 0$ , so  $K_0 > C_0/\eta a$ .
    # On  $[\text{rho}_c, 1/2]$ ,  $\eta a = \omega$  and  $k_{11}$  is increasing. On  $[1/2, 1]$ ,
    #  $d(\eta a^{k_{11}})/d \text{rho} < 0$  follows from  $\arccos \text{rho} = \sqrt{2(1-\text{rho})}$  and
    #  $\sqrt{2(1+\text{rho})} < \sqrt{2}$ . The only finite endpoint check is therefore  $\text{rho}=1/2$ .
    k11_half = sqrt_interval(4 * PI**2 + 132)
    central_endpoint_pass = (constants["omega"] * k11_half).hi < constants["C0"].lo
    ledger.require(
        central_endpoint_pass,
        " $\eta a^{k_{11}} < C_0$  at  $\text{rho}=1/2$ ",
    )
    ledger.require(1 + Q(1, 2) < 2, "K0 monotonic reduction  $\sqrt{1+\text{rho}} < \sqrt{2}$ ")
    ledger.require(
        constants["C0"].lo > 0 and constants["omega"].lo > 0,
        "K0 positive-root identity gives  $K_0 > C_0/\eta a$ ",
    )

    # For  $\text{rho} \geq 5/6$ , also beat the seam  $54/(1-\text{rho})^2$ . Squaring and taking

```

```

# s=1-rho<=1/6 leaves pi^2 s^2+132 s^4<2916.
seam_lhs = PI**2 / 36 + Q(132, 6**4)
seam_pass = seam_lhs.hi < 54**2
ledger.require(seam_pass, "k11 below seam-54 branch")
ledger.require(constants["rho_c"].hi < Q(39, 50) < Q(5, 6), "K0-only interval avoids seam")
ledger.require(constants["omega"].hi < constants["rho_c"].lo, "K0-only interval avoids H0")
ledger.require(
    constants["omega"].hi < constants["rho_c"].lo
    and Q(39, 50) < Q(5, 6),
    "U=K0 on rho_c<=rho<39/50",
)
ledger.require(central_endpoint_pass and seam_pass, "k11<U on rho_c<=rho<7/8")

def verify_product_deficit(ledger: Ledger) -> None:
    c_d = Q(1382, 3125)
    a_coeff = Q(1, 2) - c_d
    ledger.require(a_coeff > 0, "product deficit positive N^2 coefficient")

    # B(t)=t^2-2Ct+1/6 has minimum 1/6-C^2. This and
    # A(2N+1) show F(N+1,t)-F(N,t)>0 for N>=1.
    b_min = Q(1, 6) - c_d * c_d
    ledger.require(3 * a_coeff + b_min > 0, "product deficit increases in N")

    # At N=1, F'(t)=2(t-C)(t+1), so the minimum on 0<t<=1 is t=C.
    minimum = (
        Q(2, 3) * c_d**3
        + (1 - c_d) * c_d**2
        - 2 * c_d**2
        + Q(2, 3)
        - c_d
    )
    ledger.require(minimum == Q(1822532, 91552734375), "exact product-deficit minimum")
    ledger.require(minimum > 0, "D(a)>1382/3125 a^2 for all N,t")
    ledger.require(0 < c_d < 1, "product critical point lies in 0<t<=1")

    # P4 itself follows from exact sums of n^2. Check a finite symbolic grid
    # of indeterminates represented by rational N,t; this verifies the algebraic
    # identity, while the formula below is used for all N.
    for n in range(1, 7):
        for t in (Q(1, 7), Q(1, 2), Q(1)):
            sum_n2 = Q(n * (n + 1) * (2 * n + 1), 6)
            x = n + t
            direct = Q(2, 3) * x**3 - n * x**2 + sum_n2
            expanded = Q(n * n, 2) + n * (t * t + Q(1, 6)) + Q(2, 3) * t**3
            ledger.require(direct == expanded, f"product deficit expansion N={n},t={t}")

def verify_optical_constants(ledger: Ledger) -> None:
    eps0 = Q(11, 50)
    c = Q(1126, 625)
    q = Q(106, 333)
    c_d = Q(1382, 3125)
    r_l = Q(39569, 2772225000)
    r_h = Q(14817541, 472867032960000)

    low_cost = Q(2, 3) * c * q + eps0 / 4 + eps0**2 * q**2
    ledger.require(c_d - low_cost == r_l, "exact low optical endpoint reserve")
    ledger.require(r_l > 0, "low optical reserve positive")
    ledger.require(
        all(coefficient > 0 for coefficient in (Q(1, 4), q**2)),
        "low optical cost increases in epsilon (positive coefficients)",
    )

    high_radial = (1 - eps0) ** 2 / 4 - Q(22, 7) * (1 - eps0) * eps0 / (4 * c)
    highAngular = (
        q * eps0
        + eps0**2 / (4 * c)
        + q * eps0**3 / (4 * c)
        + eps0**4 / (16 * c**2)
    )
    ledger.require(high_radial - highAngular == r_h, "exact high optical endpoint reserve")
    ledger.require(r_h > 0, "high optical reserve positive")

    # L'(eps)<0 because eps<=11/50<1/2; E'(eps)>0 coefficientwise.
    ledger.require(eps0 < Q(1, 2), "high radial screen decreases on epsilon interval")
    ledger.require(
        all(
            coefficient > 0
            for coefficient in (q, Q(1, 4) / c, q / (4 * c), Q(1, 16) / c**2)
        ),
        "high angular screen increases in epsilon (positive coefficients)",
    )

    # Actual scaling implications at a<=c/eps (low) and a>=c/eps (high).
    low_cubic_actual = Q(2, 3) * c / PI
    ledger.require(
        low_cubic_actual.hi < Q(2, 3) * c * q,
        "low optical a<=c/epsilon turns cubic cost into 2cq/3",
    )
    ledger.require(
        ((1 / PI) ** 2).hi < q**2,
        "low optical a>=pi turns epsilon^2/(pi*a) into epsilon^2 q^2",
    )

    common_face = c / eps0
    angular_at_common_face = (

```

```

    eps0 / PI
    + eps0**2 / (4 * PI * common_face)
    + eps0 / (4 * common_face)
    + eps0**2 / (16 * common_face**2)
)
ledger.require(
    angular_at_common_face.hi < high_angular,
    "full high angular ceiling-error screen at a=c/epsilon",
)
ledger.require(
    all(
        coefficient > 0
        for coefficient in (
            eps0**2 / (4 * PI.lo),
            eps0 / 4,
            eps0**2 / 16,
        )
    ),
    "high angular error decreases for a>=c/epsilon",
)

radial_at_common_face = (
    (1 - eps0) ** 2 / 4
    - PI * (1 - eps0) * eps0 / (4 * c)
)
ledger.require(
    radial_at_common_face.lo > high_radial,
    "full high radial-deficit screen at a=c/epsilon",
)
ledger.require(
    (1 - eps0) * eps0 > 0,
    "high radial deficit increases for a>=c/epsilon",
)

low_face_slack = c - eps0 * common_face
high_face_slack = eps0 * common_face - c
ledger.require(
    low_face_slack == 0 and high_face_slack == 0 and eps0 > 0 and c > 0,
    "a=c/epsilon belongs to both optical pieces",
)

# Exact count/error identities at walls.
for j in range(0, 12):
    ledger.require(sum(2 * ell + 1 for ell in range(j)) == j * j, f"angular ceiling block j={j}")
ledger.require(
    strict_positive_integer_count(Q(1)) == 0,
    "a=pi radial wall uses (N,t)=(m-1,1) and is excluded",
)
verify_product_deficit(ledger)

def verify_exact_subtraction(ledger: Ledger) -> None:
    rho_states = (
        "rho_star", "low1", "rho0", "low2", "rho_c", "pre_opt", "opt_face",
        "post_opt", "rho_7_8",
    )
    k_states = ("below_k6", "k6", "stair", "k11", "post_k11", "U", "above_U")

    def low_d19(rho: str) -> bool:
        return rho in {"low1", "rho0", "low2"}

    def high_ratio(rho: str) -> bool:
        return rho in {"rho_c", "pre_opt", "opt_face", "post_opt"}

    def d19(rho: str, k: str) -> bool:
        if low_d19(rho):
            return k not in {"U", "above_U"}
        return high_ratio(rho) and k in {"stair", "k11", "post_k11"}

    def lower_cover(rho: str, k: str) -> bool:
        return low_d19(rho) and k not in {"U", "above_U"}

    def stair(rho: str, k: str) -> bool:
        return high_ratio(rho) and k in {"stair", "k11"}

    def optical(rho: str, k: str) -> bool:
        return rho in {"opt_face", "post_opt"} and k == "post_k11"

    def d20(rho: str, k: str) -> bool:
        return rho in {"rho_c", "pre_opt"} and k == "post_k11"

    for rho in rho_states:
        for k in k_states:
            remainder = d19(rho, k) and not (lower_cover(rho, k) or stair(rho, k) or optical(rho, k))
            ledger.require(remainder == d20(rho, k), f"D19 subtraction face rho={rho},K={k}")

    ledger.require(
        d19("rho0", "post_k11") and lower_cover("rho0", "post_k11"),
        "rho0 fiber belongs to lower cover and is removed once",
    )
    ledger.require(
        d20("rho_c", "post_k11") and not optical("rho_c", "post_k11"),
        "rho_c belongs to D20 ratio face after closed stair removal",
    )
    ledger.require(
        optical("opt_face", "post_k11") and not d20("opt_face", "post_k11"),
        "rho=39/50 optical face is removed inclusively",
    )

```

```

)
ledger.require(
  stair("rho_c", "k11") and not d19("rho_c", "U"),
  "K=k11 is stair-owned; K=U is inherited-owner excluded",
)
ledger.require(
  d19("pre_opt", "post_k11")
  and not lower_cover("pre_opt", "post_k11")
  and not stair("pre_opt", "post_k11")
  and not optical("pre_opt", "post_k11")
  and d20("pre_opt", "post_k11"),
  "pre-optical post-k11 cell survives exactly as D20",
)

def verify_seam_ledger(ledger: Ledger) -> dict[str, Q]:
  """Exact seam derivative, initial reserve, and terminal payments."""

  pi_upper = Q(1571, 500)
  sqrt2_lower = Q(140, 99)
  sqrt2_upper = Q(99, 70)
  ledger.require(Pi.hi < pi_upper, "seam Machin enclosure pi<1571/500")
  ledger.require(sqrt2_lower**2 < 2, "seam 140/99<sqrt(2)")
  ledger.require(sqrt2_upper**2 > 2, "seam sqrt(2)<99/70")

  epsilon = Q(1, 6)
  rho = Q(5, 6)
  kappa = Q(54)
  shelf = Q(13, 5)
  displacement_cap = Q(159, 400)
  loss = Q(14, 3)
  y_upper = Q(49, 120)
  ledger.require(y_upper**2 - epsilon == Q(1, 14400) > 0,
    "seam y<49/120 by positive squaring")
  ledger.require(shelf**2 - 2 * pi_upper == Q(119, 250) > 0,
    "seam shelf sqrt(2*pi*rho)<13/5")

  displacement_first = epsilon**2 / (kappa * rho)
  radical_upper = Q(763, 5000)
  radical_square = pi_upper / 135
  ledger.require(displacement_first == Q(1, 1620),
    "seam first displacement term")
  ledger.require(radical_upper**2 - radical_square
    == Q(8563, 675000000) > 0,
    "seam radical enclosure 763/5000")
  displacement = displacement_first + shelf * radical_upper
  ledger.require(displacement == Q(804689, 2025000),
    "seam displacement proxy exact value")
  ledger.require(displacement_cap - displacement == Q(497, 4050000) > 0,
    "seam displacement below 159/400")
  ledger.require(rho * (1 - displacement_cap) == Q(241, 480) > Q(1, 2),
    "seam localization x0/K>241/480>1/2")

  a_upper = epsilon / rho
  dqdp = shelf * y_upper / kappa
  derivative_cap = (
    2 * pi_upper**2 * dqdp
    * (1 + displacement_cap + 2 * a_upper)
    / (1 - displacement_cap) ** 3
  )
  derivative_target = Q(2260740364246, 708624500625)
  derivative_margin = Q(16, 5) - derivative_cap
  ledger.require(dqdp == Q(637, 32400), "seam exact dq/dP cap")
  ledger.require(derivative_cap == derivative_target,
    "seam exact partial_P H upper proxy")
  ledger.require(derivative_margin
    == Q(6858037754, 708624500625) > 0,
    "seam partial_P H<16/5")
  ledger.require(2 * loss - Q(16, 5) == Q(92, 15) > 0,
    "seam synthetic slope reserve 92/15")

  endpoint_q = Q(10093, 9720)
  endpoint_qhat = Q(373, 48600)
  ledger.require(endpoint_q - 1 == Q(373, 9720),
    "seam endpoint Q-1")
  ledger.require(a_upper * (endpoint_q - 1) == endpoint_qhat,
    "seam endpoint qhat")
  initial_reserve = (
    loss**2
    - 2 * pi_upper**2 * endpoint_q / (1 - endpoint_qhat) ** 2
  )
  ledger.require(initial_reserve
    == Q(2505132463469, 2616573970125) > 0,
    "seam initial B^2-H(B,B) reserve")

  gain = 2 * sqrt2_lower * kappa / (3 * pi_upper)
  gain_minus_loss = gain - loss
  inverse_y = Q(120, 49)
  interface = Q(8, 15) * sqrt2_upper
  dangerous = (gain_minus_loss * inverse_y - 1) - interface
  safe = (gain * inverse_y - 1) - interface
  ledger.require(gain == Q(280000, 17281), "seam action gain coefficient")
  ledger.require(gain_minus_loss == Q(598066, 51843),
    "seam dangerous-branch gain after R loss")
  ledger.require(interface == Q(132, 175), "seam interface remainder cap")
  ledger.require(dangerous == Q(80132733, 3024175) > 0,
    "seam dangerous-branch terminal reserve")

```



```

ledger.require(safe == Q(114694733, 3024175) > 0,
               "seam safe-branch terminal reserve")
return {
  "derivative_cap": derivative_cap,
  "derivative_margin": derivative_margin,
  "initial_reserve": initial_reserve,
  "dangerous_reserve": dangerous,
  "safe_reserve": safe,
}

def verify_k6_and_lower_d_ledgers(ledger: Ledger) -> dict[str, Q]:
    """Map every printed k6 and lower-d reserve to its wall and cap."""

    pi_lower = Q(333, 106)
    pi_upper = Q(22, 7)
    ledger.require(PI.lo > pi_lower, "payment ledger pi>333/106")
    ledger.require(PI.hi < pi_upper, "payment ledger pi<22/7")

    moving_cases = (
        ("k1", Q(1, 8), 1, 2, 4,
         Q(153864824754340538785, 348796101192617852928)),
        ("k2", Q(3, 10), 1, 6, 9,
         Q(1399810848073457853053, 394237491401523000000)),
        ("k3", Q(1, 2), 1, 12, 16,
         Q(137027505353736631, 514922437748928)),
        ("k4", Q(1, 2), 1, 20, 25,
         Q(2507119282010251109, 13902905819221056)),
        ("k5", Q(13, 25), 1, 30, 36,
         Q(92672404686738742434741, 709259556128000000000)),
        ("p1", Q(1, 5), 4, 2, 29,
         Q(373255772037478993002833, 868931613701316000000)),
    )
    for name, rho, a, b, cap, expected in moving_cases:
        A = a * pi_lower**2 / (1 - rho) ** 2 + b
        B = Q(7, 99) * (1 - rho**3)
        delta = B**2 * A**3 - cap**2
        ledger.require(delta == expected > 0,
                       f"k6 moving wall {name} at rho={rho} pays cap {cap}")

    fixed_cases = (
        ("k2", Q(3, 10), Q(51, 10), 9, Q(1387329, 110000000)),
        ("k3", Q(1, 2), Q(13, 2), 16, Q(6277, 6336)),
        ("k4", Q(1, 2), Q(15, 2), 25, Q(775, 704)),
        ("k5", Q(13, 25), Q(17, 2), 36, Q(452843, 343750)),
        ("p1", Q(1, 5), Q(77, 10), 29, Q(849901, 281250)),
    )
    for name, rho, wall, cap, expected in fixed_cases:
        payment = Q(2, 9) * Q(7, 22) * (1 - rho**3) * wall**3
        ledger.require(payment - cap == expected > 0,
                       f"k6 fixed wall {name} at rho={rho} pays cap {cap}")

    lower_caps = (
        ("L<K<=4", 1, 1),
        ("4<K<=51/10", 1 + 3, 4),
        ("51/10<K<=13/2,K<=2z", 1 + 3 + 5, 9),
        ("51/10<K<=13/2,K>2z", 1 + 3 + 5 + 1, 10),
        ("13/2<K<=15/2,K<=2z", 1 + 3 + 5 + 7, 16),
        ("13/2<K<=15/2,K>2z", 1 + 3 + 5 + 7 + 1, 17),
        ("15/2<K<=77/10", 1 + 3 + 5 + 7 + 9 + 1, 26),
        ("77/10<K<=17/2", 1 + 3 + 5 + 7 + 9 + 1 + 3, 29),
        ("17/2<K<=9", 1 + 3 + 5 + 7 + 9 + 11 + 1 + 3, 40),
        ("9<K<=d", 1 + 3 + 5 + 7 + 9 + 11 + 1 + 3 + 5, 45),
    )
    for cell, derived, printed in lower_caps:
        ledger.require(derived == printed, f"lower-d cap cell {cell}",
                       kind="bookkeeping")

    ledger.require(4 * pi_lower > 13 * Q(18, 19),
                  "lower-d 2z>13/2 at rho0 using sqrt337<19")
    ledger.require(2 * pi_upper + 1 < Q(15, 2),
                  "lower-d 2z<15/2 at rho_c")
    ledger.require(2 * pi_lower * Q(19, 18) > Q(13, 2),
                  "lower-d cap-10 row is empty because 2z>13/2")

    return {"last_moving_delta": moving_cases[-1][-1]}

def verify_lower_d_payments(ledger: Ledger) -> dict[str, Q]:
    """Exact fixed and moving payments for the lower-d cap table."""

    coefficient = Q(7, 99) * (1 - Q(7, 50) ** 3)
    fixed = (
        (Q(4), 4, Q(793292, 1546875)),
        (Q(51, 10), 9, Q(486236661, 1375000000)),
        (Q(13, 2), 16, Q(333100003, 990000000)),
        (Q(15, 2), 26, Q(329797, 88000)),
        (Q(77, 10), 29, Q(3590476297, 1125000000)),
        (Q(17, 2), 40, Q(327078887, 990000000)),
        (Q(9), 45, Q(8805519, 1375000)),
    )
    for wall, cap, expected in fixed:
        reserve = coefficient * wall**3 - cap
        ledger.require(reserve == expected > 0,
                       f"lower-d fixed wall K={wall} pays cap {cap}")

    pi_lower = Q(333, 106)

```

```

    initial = (3 + 6 * pi_lower + 4 * pi_lower**2) / 18
    ledger.require(initial - Q(19, 6) == Q(675, 2809) > 0,
        "lower-d initial moving L payment exceeds 19/6")
    rho = Q(1, 19)
    moving_2z = Q(16, 9) * pi_lower**2 * (1 - rho**3) / (1 - rho) ** 3
    ledger.require(moving_2z - Q(508, 27) == Q(137795, 75843) > 0,
        "lower-d moving 2z payment exceeds 508/27")
    return {"initial": initial, "moving_2z": moving_2z}

def run_core_ledger() -> Ledger:
    """Regenerate the 488 substantive plus 65 bookkeeping check registry."""

    ledger = Ledger()
    verify_pi(ledger)
    constants = verify_thresholds(ledger)
    verify_strict_face_conventions(ledger)
    verify_zero_specializations(ledger)
    verify_lower_inventories_and_payments(ledger, constants)
    verify_checkpoint_inventories(ledger, constants)
    verify_cap_tables_and_payments(ledger, constants)
    verify_localizations(ledger, constants)
    verify_u_and_k11(ledger, constants)
    verify_optical_constants(ledger)
    verify_exact_subtraction(ledger)
    return ledger

def registry_digest(*registries: tuple[str, Ledger]) -> tuple[int, str]:
    rows = tuple(
        row
        for namespace, ledger in registries
        for row in ledger.canonical_rows(namespace)
    )
    payload = ("\n".join(rows) + "\n").encode("ascii")
    return len(rows), hashlib.sha256(payload).hexdigest()

def main() -> int:
    try:
        core = run_core_ledger()
        supplement = Ledger()
        seam = verify_seam_ledger(supplement)
        verify_k6_and_lower_d_ledgers(supplement)
        lower = verify_lower_d_payments(supplement)
        supplement.require(Q(2409, 100) - Q(70, 3)
            == Q(227, 300) > 0,
            "cap74 ball wall and product wall are incompatible")
        rows, digest = registry_digest(("core", core), ("supplement", supplement))
    except (VerificationError, ValueError, ZeroDivisionError) as exc:
        print("FAIL")
        print(f"first issue: {exc}")
        return 1

    counts = core.counts()
    print("PASS")
    print("first issue: none")
    print(f"core exact finite checks: {len(core.labels)}")
    print(f"core substantive checks: {counts['substantive']}")
    print(f"core bookkeeping identities: {counts['bookkeeping']}")
    print(f"supplemental exact checks: {len(supplement.labels)}")
    print(f"combined registry rows: {rows}")
    print(f"canonical registry sha256: {digest}")
    print(f"seam partial_P H cap: {seam['derivative_cap']}")
    print(f"seam partial_P H margin to 16/5: {seam['derivative_margin']}")
    print(f"seam initial reserve: {seam['initial_reserve']}")
    print(f"lower-d moving 2z lower bound: {lower['moving_2z']}")
    print("cap-74 incompatibility gap: 227/300")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

S2 Executable interval certificates

The following two programs are complete: they read no input files and require only Python 3 and `python-flint`. Save each listing under the filename printed immediately above it and run

```

python standalone_round21_compact_proxy.py --precision-bits 256
python standalone_round21_compact_proxy.py --precision-bits 384
python standalone_round21_aggregate_tail.py --precision 192
python standalone_round21_aggregate_tail.py --precision 384

```

A successful run prints PASS. Any failed partition, precision, or outward-interval proof predicate gives a nonzero exit status.

The recorded environment was Python 3.14.4 with `python-flint` 0.8.0 and FLINT 3.3.1. These versions identify the replay; the proof predicates are the exact and outward-interval comparisons printed below.

S2.1 Compact-proxy rectangle certifier

`standalone_round21_compact_proxy.py`

```

1  #!/usr/bin/env python3
   """Standalone finite Arb certificate for the compact shell rectangle.

   The program certifies that the coarse integer proxy is strictly below the
   three-dimensional Weyl term on
6
   7/51 <= rho <= 39/50,    12 <= K <= 200.

   It reads no files. All geometry and branch decisions are exact ‘‘Fraction’’
   operations. Arb supplies outward transcendental enclosures. Floating-point
11 numbers choose only between two legal subdivision directions and provide
   display diagnostics; they never decide whether a box passes.

   Dependency: python-flint. Run with
16 python standalone_round21_compact_proxy.py
   """

   from __future__ import annotations

21 import argparse
   import hashlib
   import sys
   from dataclasses import dataclass, replace
   from fractions import Fraction
26 from typing import Iterable, Sequence

   try:
       from flint import arb, ctx, fmpq
   except ImportError as exc: # pragma: no cover - exercised only on a bad host
31     raise SystemExit("python-flint is required for this certificate") from exc

   SCHEMA = "standalone-round21-compact-proxy-v1-half-open-faces"
   MIN_PRECISION_BITS = 192
36 DEFAULT_PRECISION_BITS = 256
   DEFAULT_MAX_DEPTH = 30
   DEFAULT_MAX_BOXES = 100_000

41 class CertificateError(RuntimeError):
   """The first rigorous certificate condition that failed."""

   @dataclass(frozen=True, order=True)
46 class Box:
       rho_lo: Fraction
       rho_hi: Fraction
       k_lo: Fraction
       k_hi: Fraction
51
       def area(self) -> Fraction:
           return (self.rho_hi - self.rho_lo) * (self.k_hi - self.k_lo)

56 ROOT_BOX = Box(Fraction(7, 51), Fraction(39, 50), Fraction(12), Fraction(200))

   @dataclass(frozen=True)

```

```

class FloorUpper:
61     count: int
        ceiling: int
        wall_straddle: bool

66 @dataclass(frozen=True)
class ProxyUpper:
        value: int
        comparisons: int
        wall_straddles: int

71

@dataclass(frozen=True)
class BoxEvaluation:
76     proxy_upper: int
        comparison: arb
        passed: bool

        def subdivision_score(self) -> float:
81         """A non-proof heuristic; PASS never depends on this float."""

            return float(self.comparison.mid())

@dataclass(frozen=True)
86 class Leaf:
        box: Box
        depth: int
        proxy_upper: int

91

@dataclass(frozen=True)
class Certificate:
        leaves: tuple[Leaf, ...]
        max_depth: int
96     precision_bits: int
        min_gap_display: float
        digest: str
        generated_proxy_corners: int
        generated_floor_comparisons: int
101    generated_wall_straddles: int

        def fraction_text(value: Fraction) -> str:
            if value.denominator == 1:
106                 return str(value.numerator)
            return f"{value.numerator}/{value.denominator}"

        def fraction_to_arb(value: Fraction) -> arb:
111             return arb(fmpq(value.numerator, value.denominator))

        def active_channel_count(k: Fraction) -> int:
116         """Number of half-integers  $nu=ell+1/2$  satisfying the strict cutoff  $nu < K$ ."""

            shifted = k - Fraction(1, 2)
            if shifted <= 0:
                return 0
            return (shifted.numerator + shifted.denominator - 1) // shifted.denominator

121

        def g_arb(a: Fraction, nu: Fraction) -> arb:
            r"""Outward Arb enclosure of

126             
$$G_a(nu) = \begin{cases} (\sqrt{a^2 - nu^2} - nu \cdot \text{acos}(nu/a)) / \pi & \text{if } 0 \leq nu < a, \\ 0 & \text{if } nu \geq a. \end{cases}$$


            Exact rational comparisons own both interfaces. In particular, ‘‘nu=a’’
            returns exact zero and never asks Arb to resolve ‘‘sqrt(0)’’ or ‘‘acos(1)’’.
131         """

```

```

136     if a <= 0 or nu < 0:
        raise ValueError("G requires a>0 and nu>=0")
    if nu >= a:
        return arb(0)
    aa = fraction_to_arb(a)
    vv = fraction_to_arb(nu)
    radicand = aa * aa - vv * vv
    if not (radicand > 0):
141         raise CertificateError(f"Arb failed to prove positive G radicand at a={a}, nu={nu}")
    return (radicand.sqrt() - vv * (vv / aa).acos()) / arb.pi()

def proxy_argument_arb(rho: Fraction, k: Fraction, nu: Fraction) -> arb:
146     """Outward enclosure of  $G_K(nu) - G_{\rho K}(nu) + 1/4$ ."""

    if not (0 < rho < 1 and k > 0 and nu >= 0):
        raise ValueError("proxy point is outside 0<rho<1, K>0, nu>=0")
    inner = rho * k
151     return g_arb(k, nu) - g_arb(inner, nu) + fraction_to_arb(Fraction(1, 4))

def strict_count_upper_from_arb(value: arb) -> FloorUpper:
    r"""Certify an upper bound for the strict positive-integer count [value]_<.

156     If Arb proves ' $value \leq M$ ', every positive integer strictly below value
    is at most ' $M-1$ '. At exact value 3 this returns 2. If an inexact ball
    straddles an integer wall, the next integer is used, giving a safe one-unit
    overestimate instead of making an uncertified floor decision.

161     """

    if not value.is_finite():
        raise CertificateError("non-finite Arb proxy argument")
    if not (value > 0):
166         raise CertificateError(f"Arb failed to prove positive proxy argument: {value}")
    ceiling_ball = value.upper().ceil()
    ceiling_fmpz = ceiling_ball.unique_fmpz()
    if ceiling_fmpz is None:
        raise CertificateError(f"could not extract an exact integer ceiling from {value}")
171     ceiling = int(ceiling_fmpz)
    if not (value <= arb(ceiling)):
        raise CertificateError(f"Arb did not certify proxy argument <= {ceiling}: {value}")
    count = max(0, ceiling - 1)
    wall_straddle = count > 0 and value.contains(count) and not value.is_exact()
176     return FloorUpper(count=count, ceiling=ceiling, wall_straddle=wall_straddle)

def coarse_proxy_upper(rho: Fraction, k: Fraction) -> ProxyUpper:
    r"""Certify the weighted coarse proxy from per-channel Arb enclosures."""
181

    total = 0
    comparisons = 0
    wall_straddles = 0
    for ell in range(active_channel_count(k)):
186         nu = Fraction(2 * ell + 1, 2)
        if not (nu < k):
            raise CertificateError("strict active-channel enumeration included nu=K")
            bound = strict_count_upper_from_arb(proxy_argument_arb(rho, k, nu))
            total += (2 * ell + 1) * bound.count
191         comparisons += 1
        wall_straddles += int(bound.wall_straddle)
    return ProxyUpper(total, comparisons, wall_straddles)

196 def weyl_arb(rho: Fraction, k: Fraction) -> arb:
    r"""Outward Arb enclosure of  $2(1-\rho^3)K^3/(9\pi)$ ."""

    rr = fraction_to_arb(rho)
    kk = fraction_to_arb(k)
201     return arb(2) * (arb(1) - rr**3) * kk**3 / (arb(9) * arb.pi())

```

```

class Certifier:
    """Cached 256-bit (or stronger) corner evaluator and adaptive prover."""
206
    def __init__(
        self,
        precision_bits: int = DEFAULT_PRECISION_BITS,
        max_depth: int = DEFAULT_MAX_DEPTH,
211
        max_boxes: int = DEFAULT_MAX_BOXES,
    ) -> None:
        if precision_bits < MIN_PRECISION_BITS:
            raise CertificateError(
                f"precision {precision_bits} is below the required {MIN_PRECISION_BITS} bits"
216
            )
        if max_depth < 1 or max_boxes < 1:
            raise ValueError("max_depth and max_boxes must be positive")
        self.precision_bits = precision_bits
        self.max_depth = max_depth
221
        self.max_boxes = max_boxes
        self._proxy_cache: dict[tuple[Fraction, Fraction], ProxyUpper] = {}
        self._weyl_cache: dict[tuple[Fraction, Fraction], arb] = {}

    def proxy_upper(self, rho: Fraction, k: Fraction) -> ProxyUpper:
226
        key = (rho, k)
        if key not in self._proxy_cache:
            with ctx.workprec(self.precision_bits):
                self._proxy_cache[key] = coarse_proxy_upper(rho, k)
            return self._proxy_cache[key]
231

    def weyl(self, rho: Fraction, k: Fraction) -> arb:
        key = (rho, k)
        if key not in self._weyl_cache:
            with ctx.workprec(self.precision_bits):
236
                self._weyl_cache[key] = weyl_arb(rho, k)
            return self._weyl_cache[key]

    def evaluate(self, box: Box) -> BoxEvaluation:
        """Use two monotone corners and accept only a certain Arb inequality."""
241

        proxy = self.proxy_upper(box.rho_lo, box.k_hi)
        with ctx.workprec(self.precision_bits):
            # lower() is outward; PASS means w_lower - integer is certainly > 0.
            w_lower = self.weyl(box.rho_hi, box.k_lo).lower()
246
            comparison = w_lower - proxy.value
            passed = bool(comparison > 0)
            return BoxEvaluation(proxy.value, comparison, passed)

    @staticmethod
251
    def _split_rho(box: Box) -> tuple[Box, Box]:
        mid = (box.rho_lo + box.rho_hi) / 2
        return (
            Box(box.rho_lo, mid, box.k_lo, box.k_hi),
            Box(mid, box.rho_hi, box.k_lo, box.k_hi),
256
        )

    @staticmethod
    def _split_k(box: Box) -> tuple[Box, Box]:
        mid = (box.k_lo + box.k_hi) / 2
261
        return (
            Box(box.rho_lo, box.rho_hi, box.k_lo, mid),
            Box(box.rho_lo, box.rho_hi, mid, box.k_hi),
        )

    @staticmethod
266
    def _split_score(
        evaluations: Sequence[BoxEvaluation]
    ) -> tuple[int, float, float]:
        # Floats select only a subdivision direction. Every eventual leaf is
        # independently accepted by the outward-Arb predicate above.
        displays = tuple(item.subdivision_score() for item in evaluations)
271
        return (sum(item.passed for item in evaluations), min(displays), sum(displays))

    def build(self, root: Box = ROOT_BOX) -> tuple[Leaf, ...]:
        if not (Fraction(0) < root.rho_lo < root.rho_hi < Fraction(1)):
            raise ValueError("root rho interval must lie strictly inside (0,1)")

```

```

276     if not (Fraction(0) < root.k_lo < root.k_hi):
        raise ValueError("root K interval must be positive and nondegenerate")

    stack: list[tuple[Box, int]] = [(root, 0)]
    leaves: list[Leaf] = []
281    while stack:
        box, depth = stack.pop()
        evaluation = self.evaluate(box)
        if evaluation.passed:
            leaves.append(Leaf(box, depth, evaluation.proxy_upper))
286            continue
        if depth >= self.max_depth:
            raise CertificateError(
                f"depth limit {self.max_depth} at failed box {box}; "
                f"comparison={evaluation.comparison}"
            )
291        if len(leaves) + len(stack) + 2 > self.max_boxes:
            raise CertificateError(
                f"box-count limit {self.max_boxes} before splitting failed box {box}"
            )

296        rho_children = self._split_rho(box)
        k_children = self._split_k(box)
        rho_evaluations = tuple(self.evaluate(child) for child in rho_children)
        k_evaluations = tuple(self.evaluate(child) for child in k_children)
301        if self._split_score(rho_evaluations) > self._split_score(k_evaluations):
            chosen = rho_children
        else:
            chosen = k_children
            # Reverse push makes the lower child the deterministic next item.
306            for child in reversed(chosen):
                stack.append((child, depth + 1))
    return tuple(leaves)

@property
311 def proxy_corner_count(self) -> int:
    return len(self._proxy_cache)

@property
def floor_comparison_count(self) -> int:
316     return sum(item.comparisons for item in self._proxy_cache.values())

@property
def wall_straddle_count(self) -> int:
321     return sum(item.wall_straddles for item in self._proxy_cache.values())

def coordinate_owned(value: Fraction, lo: Fraction, hi: Fraction, root_hi: Fraction) -> bool:
    """Lower-closed/upper-open ownership, closing only the root upper face."""

326     return lo <= value and (value < hi or (value == root_hi and hi == root_hi))

def box_owns(root: Box, box: Box, rho: Fraction, k: Fraction) -> bool:
    """Unique shared-face ownership convention used by the finite partition."""

331     return coordinate_owned(rho, box.rho_lo, box.rho_hi, root.rho_hi) and coordinate_owned(
        k, box.k_lo, box.k_hi, root.k_hi
    )

336 def _check_owned_vertical_face(root: Box, leaves: Sequence[Leaf], rho: Fraction) -> None:
    """Check exact, unique K-tiling by the leaves that own this rho face."""

    intervals = sorted(
341         (leaf.box.k_lo, leaf.box.k_hi)
        for leaf in leaves
        if coordinate_owned(rho, leaf.box.rho_lo, leaf.box.rho_hi, root.rho_hi)
    )
    if not intervals:
346         raise CertificateError(f"uncovered vertical face rho={rho}")
    cursor = root.k_lo

```

```

    for lo, hi in intervals:
        if lo < cursor:
            raise CertificateError(f"owned-face overlap at rho={rho}, K={lo}..{cursor}")
351         if lo > cursor:
            raise CertificateError(f"gap on vertical face rho={rho} at K={cursor}..{lo}")
            cursor = hi
        if cursor != root.k_hi:
            raise CertificateError(f"vertical face rho={rho} ends at K={cursor}, not {root.k_hi}")
356

def validate_partition(root: Box, leaves: Sequence[Leaf]) -> None:
    """Exact full-coverage/disjoint-interior sweep for rational rectangles."""

361     if not leaves:
        raise CertificateError("empty leaf list")
    for index, leaf in enumerate(leaves):
        box = leaf.box
        if not (
366             root.rho_lo <= box.rho_lo < box.rho_hi <= root.rho_hi
            and root.k_lo <= box.k_lo < box.k_hi <= root.k_hi
        ):
            raise CertificateError(f"leaf {index} lies outside or degenerates in root: {box}")

371     rho_faces = sorted(
        {root.rho_lo, root.rho_hi}
        | {leaf.box.rho_lo for leaf in leaves}
        | {leaf.box.rho_hi for leaf in leaves}
    )
376     for rho_lo, rho_hi in zip(rho_faces, rho_faces[1:]):
        intervals = sorted(
            (leaf.box.k_lo, leaf.box.k_hi)
            for leaf in leaves
            if leaf.box.rho_lo <= rho_lo and rho_hi <= leaf.box.rho_hi
381        )
        cursor = root.k_lo
        for k_lo, k_hi in intervals:
            if k_lo < cursor:
                raise CertificateError(
386                     f"positive-area overlap in rho slab {rho_lo}..{rho_hi} at K={k_lo}"
                )
            if k_lo > cursor:
                raise CertificateError(
                    f"coverage gap in rho slab {rho_lo}..{rho_hi}: K={cursor}..{k_lo}"
391                )
            cursor = k_hi
        if cursor != root.k_hi:
            raise CertificateError(
                f"rho slab {rho_lo}..{rho_hi} ends at K={cursor}, not {root.k_hi}"
396            )

    if sum((leaf.box.area() for leaf in leaves), Fraction(0)) != root.area():
        raise CertificateError("exact sum of leaf areas differs from root area")

401     # Closed-box estimates plus this ownership check give pointwise coverage,
    # including every shared face, not merely disjoint interiors.
    for rho in rho_faces:
        _check_owned_vertical_face(root, leaves, rho)

406

def canonical_certificate_bytes(
    root: Box,
    leaves: Iterable[Leaf],
    precision_bits: int,
411 ) -> bytes:
    """Canonical ASCII serialization of all finite certificate proof data."""

    rows = [
        SCHEMA,
416         f"precision_bits={precision_bits}",
        "root=" + ", ".join(
            fraction_text(value)
            for value in (root.rho_lo, root.rho_hi, root.k_lo, root.k_hi)
        )
    ]

```



```

    ),
421 ]
    for leaf in sorted(leaves, key=lambda item: item.box):
        box = leaf.box
        rows.append(
            "leaf="
426         + ",".join(
            fraction_text(value)
            for value in (box.rho_lo, box.rho_hi, box.k_lo, box.k_hi)
            )
        + f",depth={leaf.depth},P={leaf.proxy_upper}"
431     )
    return ("\n".join(rows) + "\n").encode("ascii")

def certificate_digest(
436     root: Box,
    leaves: Iterable[Leaf],
    precision_bits: int,
) -> str:
    return hashlib.sha256(
441         canonical_certificate_bytes(root, leaves, precision_bits)
    ).hexdigest()

def validate_leaves(certifier: Certifier, leaves: Sequence[Leaf]) -> float:
446     """Replay all proof predicates; return only a display approximation to min gap."""

    display_gaps: list[float] = []
    for index, leaf in enumerate(leaves):
        evaluation = certifier.evaluate(leaf.box)
451         if evaluation.proxy_upper != leaf.proxy_upper:
            raise CertificateError(
                f"leaf {index} proxy tampering/mismatch: stored {leaf.proxy_upper}, "
                f"recomputed {evaluation.proxy_upper}"
            )
456         if not evaluation.passed:
            raise CertificateError(
                f"leaf {index} fails certain Arb comparison: {evaluation.comparison}"
            )
        display_gaps.append(evaluation.subdivision_score())
461     return min(display_gaps)

def build_certificate(
    precision_bits: int = DEFAULT_PRECISION_BITS,
466     max_depth: int = DEFAULT_MAX_DEPTH,
    max_boxes: int = DEFAULT_MAX_BOXES,
) -> Certificate:
    certifier = Certifier(precision_bits, max_depth, max_boxes)
    leaves = certifier.build(ROOT_BOX)
471     validate_partition(ROOT_BOX, leaves)
    min_gap_display = validate_leaves(certifier, leaves)
    digest = certificate_digest(ROOT_BOX, leaves, precision_bits)
    return Certificate(
        leaves=leaves,
476         max_depth=max(leaf.depth for leaf in leaves),
        precision_bits=precision_bits,
        min_gap_display=min_gap_display,
        digest=digest,
        generated_proxy_corners=certifier.proxy_corner_count,
481         generated_floor_comparisons=certifier.floor_comparison_count,
        generated_wall_straddles=certifier.wall_straddle_count,
    )

486 def tampered_leaf(leaf: Leaf) -> Leaf:
    """Return a one-count corruption for independent replay tests."""

    return replace(leaf, proxy_upper=leaf.proxy_upper + 1)

```

```

def _parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--precision-bits", type=int, default=DEFAULT_PRECISION_BITS)
    parser.add_argument("--max-depth", type=int, default=DEFAULT_MAX_DEPTH)
496 parser.add_argument("--max-boxes", type=int, default=DEFAULT_MAX_BOXES)
    return parser

def main(argv: Sequence[str] | None = None) -> int:
501 args = _parser().parse_args(argv)
    try:
        certificate = build_certificate(
            precision_bits=args.precision_bits,
            max_depth=args.max_depth,
506 max_boxes=args.max_boxes,
        )
    except Exception as exc:
        print("STANDALONE_COMPACT_PROXY FAIL")
        print(f"first_issue={type(exc).__name__}: {exc}")
511 return 1

    print("STANDALONE_COMPACT_PROXY PASS")
    print(f"precision_bits={certificate.precision_bits}")
    print("root_rho=[7/51,39/50]")
516 print("root_K=[12,200]")
    print(f"leaf_boxes={len(certificate.leaves)}")
    print(f"max_depth={certificate.max_depth}")
    print("coverage=PASS (exact rational slab/face sweep and exact area)")
    print("nonoverlap=PASS (exact half-open shared-face ownership)")
521 print(f"generated_proxy_corners={certificate.generated_proxy_corners}")
    print(f"generated_floor_comparisons={certificate.generated_floor_comparisons}")
    print(f"generated_wall_straddles={certificate.generated_wall_straddles}")
    print(f"min_gap_lower_display={certificate.min_gap_display:.17g}")
    print(f"certificate_sha256={certificate.digest}")
526 return 0

if __name__ == "__main__":
    sys.exit(main())

```

S2.2 Aggregate-tail certifier

standalone_round21_aggregate_tail.py

```

#!/usr/bin/env python3
"""Standalone outward-Arb certificate for aggregate-tail positivity.

The proof-sign decisions use only exact 'Fraction'/'fmpq' data and
5 python-flint 'arb' interval comparisons. Decimal lower bounds in the
command-line output are display-only diagnostics. The program reads no
files and has no dependency other than Python's standard library and
python-flint.

10 Run with

    python standalone_round21_aggregate_tail.py --precision 192
"""

15 from __future__ import annotations

import argparse
import hashlib
from dataclasses import dataclass
20 from fractions import Fraction
from typing import Dict, Iterable, Mapping, Sequence

try:
    from flint import arb, ctx, fmpq
25 except ImportError as exc: # pragma: no cover - exercised only on a bad host
    raise SystemExit("python-flint is required for this certificate") from exc

```

```

SCHEMA = "standalone-round21-aggregate-tail-v1"
30 MIN_PRECISION_BITS = 192
    KO = 200
    LOWER_RHO = Fraction(7, 51)
    SPLIT_RHO = Fraction(1, 2)
    UPPER_RHO = Fraction(39, 50)
35 MAX_BOX_WIDTH = Fraction(1, 2000)
    EXPECTED_BOX_COUNT = 1286

class VerificationError(RuntimeError):
40     """Raised when an exact-partition or interval proof obligation fails."""

    @dataclass(frozen=True)
    class RationalBox:
45         lo: Fraction
         hi: Fraction
         eta_regime: str

        @property
50         def width(self) -> Fraction:
             return self.hi - self.lo

    @dataclass(frozen=True)
55     class DisplayMinimum:
         """A nonproof decimal summary of already-proved positive Arb balls."""

         approximate_lower: float
         box: RationalBox
60         ball: arb

    @dataclass(frozen=True)
    class CertificateResult:
65         precision_bits: int
         box_count: int
         low_regime_boxes: int
         high_regime_boxes: int
         digest: str
70         display_minima: Mapping[str, DisplayMinimum]

    def build_boxes() -> tuple[RationalBox, ...]:
75         """Build the exact rational partition, split at rho=1/2."""

         boxes: list[RationalBox] = []
         segments = (
             (LOWER_RHO, SPLIT_RHO, "rho_le_half"),
             (SPLIT_RHO, UPPER_RHO, "rho_ge_half"),
80         )
         for start, stop, regime in segments:
             lo = start
             while lo < stop:
                 hi = min(lo + MAX_BOX_WIDTH, stop)
                 boxes.append(RationalBox(lo, hi, regime))
85                 lo = hi
             validate_partition(boxes)
             return tuple(boxes)

90     def validate_partition(boxes: Sequence[RationalBox]) -> None:
         """Prove exact coverage, adjacency, widths, and regime ownership."""

         if len(boxes) != EXPECTED_BOX_COUNT:
95             raise VerificationError(
                 f"wrong box count: expected {EXPECTED_BOX_COUNT}, got {len(boxes)}"
             )
         if boxes[0].lo != LOWER_RHO or boxes[-1].hi != UPPER_RHO:

```

```

    raise VerificationError("box partition has the wrong outer endpoints")
100 for index, box in enumerate(boxes):
    if not (Fraction(0) < box.width <= MAX_BOX_WIDTH):
        raise VerificationError(f"invalid exact width in box {index}: {box.width}")
    if index and boxes[index - 1].hi != box.lo:
        raise VerificationError(f"gap or overlap before box {index}")
105 if box.eta_regime == "rho_le_half":
    if box.hi > SPLIT_RHO:
        raise VerificationError(f"low-eta box {index} crosses rho=1/2")
    elif box.eta_regime == "rho_ge_half":
        if box.lo < SPLIT_RHO:
110         raise VerificationError(f"high-eta box {index} crosses rho=1/2")
    else:
        raise VerificationError(f"unknown eta regime in box {index}")
    low_count = sum(box.eta_regime == "rho_le_half" for box in boxes)
    high_count = sum(box.eta_regime == "rho_ge_half" for box in boxes)
115 if (low_count, high_count) != (726, 560):
    raise VerificationError(
        f"wrong split counts: expected (726, 560), got {(low_count, high_count)}"
    )

120 def arb_of_fraction(value: Fraction) -> arb:
    return arb(fmpq(value.numerator, value.denominator))

125 def arb_box(box: RationalBox) -> arb:
    """Return an outward Arb ball covering the exact rational box."""

    midpoint = (box.lo + box.hi) / 2
    radius = (box.hi - box.lo) / 2
130 enclosure = arb(
    fmpq(midpoint.numerator, midpoint.denominator),
    fmpq(radius.numerator, radius.denominator),
    )
    # These proof checks ensure the rounded Arb representation covers both
135 # exact-rational endpoint enclosures.
    if not enclosure.contains(arb_of_fraction(box.lo)):
        raise VerificationError(f"Arb box misses exact lower endpoint {box.lo}")
    if not enclosure.contains(arb_of_fraction(box.hi)):
        raise VerificationError(f"Arb box misses exact upper endpoint {box.hi}")
140 return enclosure

def g_one(z: arb, pi: arb) -> arb:
    """Return (sqrt(1-z^2)-z*acos(z))/pi with outward Arb arithmetic."""
145 return ((arb(1) - z * z).sqrt() - z * z.acos()) / pi

def evaluate_at_k0(rho: arb, eta_regime: str) -> Mapping[str, arb]:
150 """Evaluate all base-face and K-convexity quantities on one rho ball."""

    one = arb(1)
    two = arb(2)
    half = one / two
155 k = arb(K0)
    pi = arb.pi()

    if eta_regime == "rho_le_half":
        eta = g_one(half, pi)
160 elif eta_regime == "rho_ge_half":
        eta = g_one(rho, pi)
    else:
        raise VerificationError(f"unknown eta regime: {eta_regime}")

165 d = two * rho.asin() / pi
    b = two * pi * rho / (one - rho)
    mu = rho * k
    c = b * k
    radius = mu + half
170 s = (radius * radius + c).sqrt()

```

```

p = s - radius
asinh_term = (radius / c.sqrt()).asinh()
integral = (radius * s + c * asinh_term - radius * radius) / two

175 reserve = (
    (mu - half) * (k * eta - one)
    + (d / two) * (mu - arb(3) / two) * (mu - half)
    - (one + d) * integral
    - arb(8) * (mu + half) / (arb(15) * mu.sqrt())
180 )

integral_k = rho * p + (b / two) * asinh_term
reserve_k = (
    rho * (k * eta - one)
185 + (mu - half) * eta
    + d * rho * (mu - one)
    - (one + d) * integral_k
    - two * rho * (two * mu - one) / (arb(15) * mu * mu.sqrt())
)

190 integral_kk = (
    rho * rho * (radius / s - one)
    + rho * b / (two * s)
    + b * (two * rho * k - one) / (arb(8) * k * s)
195 )
integral_kk_bound = arb(3) * b / (arb(4) * k)
interface_kk = (
    rho * rho * (two * mu - arb(3))
    / (arb(15) * mu * mu * mu.sqrt())
200 )
curvature_floor = (
    two * rho * eta
    + d * rho * rho
    - arb(3) * (one + d) * b / arb(800)
205 )
reserve_kk = (
    two * rho * eta + d * rho * rho
    - (one + d) * integral_kk + interface_kk
210 )

return {
    "eta": eta,
    "b": b,
    "mu": mu,
215 "R": radius,
    "S": s,
    "B200": reserve,
    "BK200": reserve_k,
    "F": curvature_floor,
    "I_KK": integral_kk,
220 "I_KK_bound": integral_kk_bound,
    "interface_KK": interface_kk,
    "B_KK": reserve_kk,
}

225

def _require_positive(name: str, value: arb, box: RationalBox) -> None:
    if not (value > 0):
        raise VerificationError(
230             f"could not prove {name}>0 on [{box.lo}, {box.hi}] "
             f"({box.eta_regime}): {value.str(30)}"
        )

235 def _require_less(name: str, left: arb, right: arb, box: RationalBox) -> None:
    if not (left < right):
        raise VerificationError(
            f"could not prove {name} on [{box.lo}, {box.hi}] "
            f"({box.eta_regime}): left={left.str(30)}, right={right.str(30)}"
240         )

```

```

def verify_boxes(bboxes: Iterable[RationalBox]) -> Mapping[str, DisplayMinimum]:
    """Apply every outward-Arb wall on every exact rational box."""
245
    minima: Dict[str, DisplayMinimum] = {}
    for box in bboxes:
        rho = arb_box(box)
        values = evaluate_at_k0(rho, box.eta_regime)
250
        _require_positive("1-rho", arb(1) - rho, box)
        _require_positive("mu-3/2", values["mu"] - arb(3) / arb(2), box)
        _require_positive("K*eta-1", arb(K0) * values["eta"] - arb(1), box)
        _require_positive("S-R", values["S"] - values["R"], box)
255
        _require_positive("R-rho*K", values["R"] - rho * arb(K0), box)
        _require_less(
            "I_KK < 3b/(4K)", values["I_KK"], values["I_KK_bound"], box
        )
        _require_positive("interface_KK", values["interface_KK"], box)
260
        _require_less("F < B_KK", values["F"], values["B_KK"], box)

        for name in ("B200", "BK200", "F"):
            value = values[name]
            _require_positive(name, value, box)
265
            # Selection and decimal conversion are never proof decisions.
            approximate_lower = float(value.lower())
            current = minima.get(name)
            if current is None or approximate_lower < current.approximate_lower:
                minima[name] = DisplayMinimum(approximate_lower, box, value)
270
    return minima

def _fraction_text(value: Fraction) -> str:
    return f"{value.numerator}/{value.denominator}"
275

def canonical_certificate_bytes(
    boxes: Iterable[RationalBox], precision_bits: int
) -> bytes:
280
    """Canonical ASCII serialization of the exact finite partition."""

    rows = [
        SCHEMA,
        f"precision_bits={precision_bits}",
285
        f"KO={K0}",
        f"rho={_fraction_text(LOWER_RHO)}, {_fraction_text(UPPER_RHO)}",
        f"split_rho={_fraction_text(SPLIT_RHO)}",
        f"max_box_width={_fraction_text(MAX_BOX_WIDTH)}",
    ]
290
    rows.extend(
        "box="
        + f"{_fraction_text(box.lo)}, {_fraction_text(box.hi)}, {box.eta_regime}"
        for box in boxes
    )
295
    return ("\n".join(rows) + "\n").encode("ascii")

def certificate_digest(boxes: Iterable[RationalBox], precision_bits: int) -> str:
300
    return hashlib.sha256(
        canonical_certificate_bytes(boxes, precision_bits)
    ).hexdigest()

def run_certificate(precision_bits: int = MIN_PRECISION_BITS) -> CertificateResult:
305
    if precision_bits < MIN_PRECISION_BITS:
        raise ValueError(
            f"precision must be at least {MIN_PRECISION_BITS} bits, got {precision_bits}"
        )
    ctx.prec = precision_bits
    boxes = build_boxes()
    minima = verify_boxes(boxes)
    low_count = sum(box.eta_regime == "rho_le_half" for box in boxes)
    high_count = len(boxes) - low_count
    return CertificateResult(
310

```

```

315         precision_bits=precision_bits,
           box_count=len(boxes),
           low_regime_boxes=low_count,
           high_regime_boxes=high_count,
           digest=certificate_digest(boxes, precision_bits),
320         display_minima=minima,
           )

def print_result(result: CertificateResult) -> None:
325     print("STANDALONE_AGGREGATE_TAIL PASS")
     print(f"precision_bits={result.precision_bits}")
     print(f"box_count={result.box_count}")
     print(f"low_regime_boxes={result.low_regime_boxes}")
     print(f"high_regime_boxes={result.high_regime_boxes}")
330     print("coverage=PASS (exact rational adjacency and endpoints)")
     print("proof_sign_checks=B(rho,200)>0, B_K(rho,200)>0, F(rho)>0")
     print(f"certificate_sha256={result.digest}")
     print("display_only_decimal_lower_bounds (not used for proof):")
     for name in ("B200", "BK200", "F"):
335         minimum = result.display_minima[name]
         box = minimum.box
         display = minimum.ball.lower().str(16, radius=False)
         print(
             f" {name}: {display} on intended rational box "
340             f"[{_fraction_text(box.lo)}, {_fraction_text(box.hi)}]"
         )

def main(argv: Sequence[str] | None = None) -> int:
345     parser = argparse.ArgumentParser(description=__doc__)
     parser.add_argument(
         "--precision",
         type=int,
         default=MIN_PRECISION_BITS,
350         help=f"Arb midpoint precision in bits (minimum {MIN_PRECISION_BITS})",
     )
     args = parser.parse_args(argv)
     try:
         result = run_certificate(args.precision)
355     except (VerificationError, ValueError) as exc:
         print(f"STANDALONE_AGGREGATE_TAIL FAIL: {exc}")
         return 1
     print_result(result)
     return 0
360

if __name__ == "__main__":
    raise SystemExit(main())

```

S3 Verification boundary

The exact-rational verifier uses only the Python standard library. The two interval verifiers require Python 3 and `python-flint`; their transcendental decisions use outward Arb midpoint-radius balls [1]. No displayed decimal is a proof predicate. A finite claim is accepted only by exact integer or rational arithmetic, or by a ball whose complete interval has the required strict sign.

The trusted computational boundary is therefore the Python integer/rational kernel, the `python-flint` bindings, and Arb's outward elementary functions. The partitions, strict-wall ownership rules, precision checks, coverage tests, and deterministic hashing logic are visible in the listings. For publication-level replication, reviewers should also replay the same predicates with an independently built Arb installation or a second interval library.

References

- [1] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Trans. Comput.* **66** (2017), no. 8, 1281–1292. [doi:10.1109/TC.2017.2690633](https://doi.org/10.1109/TC.2017.2690633).